

卢晓锋

个人信息

岗位：高级前端开发

经验：8 年

手机：15279189307

邮箱：luxiaofeng.code@gmail.com

学校：华东交通大学

专业：通信工程（本科）

工作经历

有赞科技有限公司	2024.06~至今	全栈开发
深圳市兔展智能科技有限公司	2020.07~2024.05	高级前端开发
深圳市秦丝科技有限公司	2019.05~2020.06	中级前端开发
深圳市猜猜城科技有限公司	2018.08~2019.05	初级前端开发

技能

- 前端**：HTML5、CSS3、JavaScript (ES6+)；**Vue**：Vue 3 升级与生态迁移、TypeScript、常用生态；**React**：业务开发与工程化实践；Webpack、Vite 构建与调试；包管理与工程化（Yarn→pnpm 等）
- 跨端**：小程序；Taro、Uni-app 等跨端框架
- 服务端与运维**：Node.js；Redis、Nginx、Docker 等常见栈
- 全栈**：Java、MySQL；业务接口与 SQL、前后端联调与问题排查
- AI 研发提效**：AI 辅助开发；**自研 MCP**（支撑完整研发工作流，提升效率）；**Cursor** 与内部 Skill（日志 / DB / 接口 / 发布等）；**OpenSpec** 规范协作；问题排查与文档沉淀

项目经历

1、AI 研发提效平台（dev-flow MCP） 有赞科技 · 2025.01 - 2026.03

项目背景：线上问题排查需在**日志、数据库、接口、发布**等多个内部系统间频繁切换，操作均在浏览器中进行，上下文割裂、路径长，严重拉低排障与发布效率。

项目描述：主导设计并开发团队内 **dev-flow MCP Server**，将**项目发布、天网日志查询、数据库查询、接口调用**等原本需在浏览器手动操作的能力，封装为 AI 助手可直接调用的 **MCP Tool**；**复用本机浏览器已登录态**，从浏览器自动延续登录，请求自动携带鉴权，无需手动复制 Token；工具在 **Cursor** 等 IDE 中串联使用，配合 **OpenSpec** 支撑需求协作与排障闭环。

业务价值：将排查与发布链路从「手动开多个 Tab → 浏览器操作 → 来回切换」压缩为在 **IDE 内一条指令完成**；日志、数据与代码改动可在同一上下文中流转，缩短排障与发布闭环，提升研发协作效率。

项目亮点：

- 研发工作流内聚到 IDE**：将日志、数据库、接口、发布等分散在多个内部系统的操作收敛为统一的 MCP Tool 调用，减少上下文切换。
- 无感登录提升可用性**：复用本机浏览器已登录会话，从浏览器自动登录延续到工具侧，无需手动复制 Token，降低接入与使用门槛。

- **Tool 粒度清晰可编排**：按单一职责拆分能力模块，便于 AI 助手基于任务步骤进行组合调用与扩展。

项目职责：主导 **MCP Server** 架构设计与核心工具开发，负责浏览器登录态衔接与鉴权、各工具输入输出协议设计、内部系统对接联调，以及团队使用规范与 Skill 文档沉淀。

技术要点：

- **MCP Server**：基于 MCP 协议设计工具集，Tool 粒度对齐单一职责，供 AI 助手按步骤编排调用
- **无感登录**：复用本机浏览器已登录会话，从浏览器自动延续登录；请求自动携带鉴权；会话失效时引导浏览器完成企业统一登录（SSO）后恢复任务
- **能力模块**：发布（触发构建/部署流程）、天网日志（按 Traceld/时间/服务名查询）、DB 查询（只读，防误操作）、接口调用（自动注入鉴权头）
- **协作串联**：与 **OpenSpec** 结合，在 IDE 内串联问题排查、需求协作与发布操作

2、用户权益次数打标 有赞科技 · 2025.10 - 2025.11

项目背景：业务需要按用户权益使用情况做**用户打标**（如分层、运营触达），并处理**过期权益**后的标签刷新；历史权益与用户数持续增长，若按「**全量重扫历史权益次数**」设计，任务规模随时间线性膨胀，不可持续。

项目难点：打标依赖累计次数，过期事件发生时需重算关联用户的次数并刷新标签；若每次都扫全量历史数据，DB 压力与任务耗时无法控制。方案上以**昨日过期**为时间窗口圈定受影响用户，增量处理替代全量扫历史。

项目描述：主导规则打标链路在 Java 侧的设计与落地；支持按**累计次数阈值或次数区间**命中规则打标；过期侧由**每日定时任务**驱动，按**昨日过期**维度圈选受影响用户集，通过**游标分批**方式重算累计次数并刷新标签，避免全量扫描历史权益数据。

业务价值：支持按规则配置用户分层与运营触达；过期侧标签可按日刷新，并以增量时间窗口替代全量扫历史，使任务规模、DB 压力与执行耗时保持可控。

项目亮点：

- **增量窗口替代全量扫描**：以**昨日过期**为时间窗口圈定受影响用户，避免任务规模随历史数据增长线性膨胀。
- **游标分批控制处理压力**：按主键游标分页拉取数据，每批处理完成后再推进 cursor，降低单次 DB 查询与应用内存压力。
- **批次内聚合减少重复打标**：按 **userId** 聚合累计次数并去重，同一用户多条权益记录合并计算，命中规则后仅触发一次打标。

项目职责：主导规则打标方案设计与 Java 侧实现，负责过期侧增量处理链路、游标分批策略、批次内聚合去重逻辑设计，并完成联调与线上问题排查。

技术要点：

- **昨日过期窗口**：MySQL 按过期时间圈选昨日过期记录，与全量历史解耦，任务规模稳定可控
- **游标分批**：按主键游标翻页，每批取 5000 条，处理完推进 cursor 再取下一批；避免全量加载导致的内存与 DB 压力
- **批次内聚合去重**：每批数据按 **userId** 聚合累计次数，同一用户多条记录自动合并为一条结果，命中阈值/区间规则后发消息触发打标，不重复发送

- **扩展性**：若单线程处理量不足，可在应用层按 `userId % N` 分片，起 `N` 个并行 worker 各自跑游标分批，`N` 可配置，核心逻辑不变

3、商家预约日历约满状态异步缓存改造

有赞科技 · 2025.02 - 2025.03

项目背景：日历需展示商家未来约 30 天左右每日是否约满；预约下单链路已有，原先列表/日历若频繁按天算约满，读路径压力大。

项目描述：在不改造预约下单主链路的前提下，通过 **NSQ** 订阅**预约成功**等相关 Topic；结合 **TSP** 定时任务，对受影响的商家 + 本地日做去重与防抖（合并短时间内的多次触发），异步重算未来约 30 天约满状态并写入 **Redis**；日历查询优先走缓存。

业务价值：将约满展示状态从实时计算改为**异步维护**，降低日历接口重复计算与读路径压力；同时保持预约下单链路不受影响，实现展示态与核心交易逻辑解耦。

项目亮点：

- **核心链路零侵入**：预约核心能力不改造，只维护展示用派生状态，避免影响交易主流程。
- **异步重算替代同步计算**：通过 **NSQ + TSP** 异步刷新约满状态，查询侧直接读缓存，降低实时计算压力。
- **一致性优先**：重算以 **DB** 为准，Redis 仅存展示结果；采用**全量重算**而非缓存内加减，避免状态漂移。

项目职责：参与异步重算方案设计与 Java 侧落地，负责 **NSQ** 消费、**TSP** 定时任务下的重算触发、去重/防抖策略、Redis Key 设计及异常兜底处理，并完成联调。

技术要点：

- **NSQ** 与预约下单解耦；**Channel** 按团队约定配置，失败与重试按现有规范兜底
- **TSP** 定时任务：对「商家 + 本地日」维度做**去重 / 防抖**，避免短时大量消息导致同一日**重复全量重算**
- **一致性取舍**：未在 **Redis** 中基于事件直接做加减，而是以 **DB** 为准全量重算，避免跨日、营业时间变更、重复消息等场景下展示状态偏差
- **Redis** 仅存约满结果；重算时查库得到当日预约与营业时间等信息，再写回缓存；键维度：**商家 + 日期**，并解析受影响本地日
- 可配合 **TTL** 或定时纠偏，控制异常场景下的缓存残留风险

4、工程化：Yarn 迁移 pnpm 与 Node 升级

有赞科技 · 2024.06 - 2024.11

项目背景：仓库长期使用 **Yarn**，随着依赖规模增长与 **Node** 版本演进，本地安装、CI 构建及多人协作环境中逐步出现依赖解析不一致、旧包兼容性差等问题，影响业务迭代与发版稳定性。

项目描述：负责推进仓库从 **Yarn** 迁移至 **pnpm**，并同步完成 **Node** 版本升级；围绕安装脚本、锁文件、CI 流水线及本地开发环境进行适配，结合 **pnpmfile** 编写兼容逻辑，处理旧依赖在新包管理机制下的解析与兼容问题，保障迁移过程平滑落地。

业务价值：统一本地与 CI 的依赖安装和构建环境，降低因依赖解析差异、Node 版本不一致导致的构建失败与发版阻塞，提升大仓场景下的工程稳定性与协作效率。

项目亮点：

- **包管理与运行时同步升级**：将 **Yarn** → **pnpm** 与 **Node** 升级协同推进，避免只升级单侧造成新的环境割裂。
- **兼容性优先的迁移策略**：通过 **pnpmfile** 对旧依赖做解析与兼容适配，降低存量包对迁移的阻力。
- **环境一致性治理**：同步调整本地开发、安装脚本与 CI 流水线，减少“本地可用、CI 失败”的情况。

项目职责：负责 **Yarn** → **pnpm** 迁移方案落地与 **Node** 升级实施，处理依赖兼容问题，调整安装脚本与 CI 配置，并协助完成本地与流水线环境对齐及问题排查。

技术要点：

- **包管理迁移**：完成 **Yarn** → **pnpm** 切换，调整锁文件、安装脚本与 CI 流程
- **Node 升级**：统一本地开发与流水线运行时版本，减少环境差异
- **兼容处理**：通过 **pnpmfile**（如 **readPackage**）对旧依赖进行兼容与解析适配，降低迁移风险

5、AI 数字人拜年小程序 兔展科技 · 2024.01 - 2024.02

项目背景：2024 年春节前约 15 天临时插入需求，**时间紧、任务重**，需在极短周期内在小程序内落地 **AI 数字人拜年互动**，支撑春节营销与品牌传播。

项目描述：担任前端负责人，推进 **AI 数字人拜年** 小程序落地：根据**组员能力**拆解前端任务并安排分工；在**需求频繁变化**的情况下**优先完成主框架搭建**，再迭代页面与交互并推进数字人联调，保障春节前发版与活动期线上稳定性。

业务价值：在春节传播窗口内提供可分享、可互动的数字人拜年体验，支撑活动侧触达与转化；在短周期、强变更约束下仍完成可交付、可迭代的**活动落地**。

项目职责：担任前端负责人，负责技术方案与任务拆解、**按成员能力分配任务与排期**、主框架与核心页面落地、跨端联调与问题闭环，协调测试与发布。

技术要点：

- **主框架优先**：需求变化下先搭建可扩展的主框架，再并行迭代业务模块与联调
- 小程序活动页、分享链路及关键交互落地
- AI 数字人能力接入、联调与体验优化
- 临期场景下的版本管理与发布节奏、线上问题跟进

项目成果：

- **时间紧任务重**下担任前端负责人，通过主框架先行与任务分工保障春节前按期上线
- 活动期保障前端侧稳定性与发版节奏

6、从 Vue1 到 Vue3：零售装修模块与工程化升级 兔展科技 · 2023.01 - 2023.07

项目背景：系统长期基于 **Vue 1** 与 **Webpack 1.x**，技术栈陈旧、构建效率偏低，已成为研发效率与交付节奏的主要制约因素。

项目描述：主导零售系统 Web 与小程序端技术栈升级，完成 **Vue 1 → Vue 3**、**Webpack 1.x → Webpack 5** 迁移，并接入 **TypeScript** 等现代化工具链；同步重构核心「装修」模块，降低老系统维护成本与迭代阻力。

业务价值：面向零售与营销侧页面、活动落地页及小程序的高频改稿与上线场景；升级后缩短从开发到联调、提测的等待周期，降低老栈带来的迭代与招聘/协作成本。

项目亮点：

- **老栈整体升级：**围绕框架、构建与类型体系统一升级，提升工程可维护性与交付效率。
- **兼容迁移提效：**通过脚本与适配层处理旧代码与路由兼容，压缩人工迁移成本。
- **核心模块重构：**针对高频变更的装修模块做重构，提升代码复用度与需求扩展效率。

项目职责：主导 **Vue 3** 升级落地、路由兼容、构建链路升级及工程化能力接入，完成核心模块重构与迁移提效脚本开发。

技术要点：

- **Vue 3 迁移：**完成语法与生态迁移，升级 **vue-loader**，推动项目由 JS 向 **TypeScript** 演进
- **构建升级：**基于 **Webpack 5** 集成 **swc**、**thread-loader** 等能力加速构建，同时兼容 **Vite**，支持体积与构建耗时分析
- **迁移提效：**编写脚本将 **js/html** 合并为 **.vue** 并做语法替换，减少重复人工迁移

项目成果：

- **Vue 3** 生态落地后，整体开发效率约提升 20%（含构建与联调等待缩短、同类需求交付周期体感等综合复盘）
- 首页装修系统重构，代码复用率提高，单文件由约 2000 行降至平均约 330 行，**装修类需求**扩展与改版成本明显下降
- **Vite** 本地构建速度提升约 90%，**Webpack** 生产构建速度提升约 36%
- 通过路由适配器平滑承接旧权限模型，路由与权限改造范围约缩小 90%

7、前端埋点架构设计 兔展科技 · 2022.04 - 2022.10

项目背景：各业务线埋点各自实现，手写比例高、易侵入业务逻辑，**缺乏可复用的分端埋点 SDK（Web / 小程序）**，需建设团队级埋点方案。

项目描述：主导 **Web** 与 **小程序** 两端埋点 SDK 的设计与落地（分端实现）；以**配置驱动 + 自动采集**覆盖点击、路由等埋点能力，减少业务侧手写与重复接入成本。

业务价值：将埋点能力沉淀为可复用的**前端埋点 SDK（分端）**，对齐团队接入约定与埋点模型，降低各项目重复开发与维护成本。

项目亮点：

- **低侵入接入：**以自动采集与配置驱动减少业务侧手写埋点，控制对业务逻辑的侵入面。
- **分端 SDK：****Web** 与 **小程序** 各自独立 SDK，能力边界与接入方式按端拆分；**配置驱动 + 自动采集**思路一致，各端分别封装路由与点击采集。
- **通用能力沉淀：**将路由与点击采集封装为可复用方案，并接入脚手架模板推动团队复用。

项目职责：主导埋点领域技术方案与**分端 SDK 架构**，定义能力边界、接入规范与扩展方式；牵头 **Web / 小程序** 端核心采集能力落地与脚手架集成；负责测试、文档、发布及业务侧接入过程中的问题闭环。

技术要点：

- **分端采集能力：****Web** 与 **小程序** 各自实现路由/页面级与点击（行为）级自动采集链路，覆盖主要埋点路径

- **配置驱动**：埋点元数据与路由、元素绑定，由配置描述，减少硬编码与手写埋点
- **低侵入与工程化**：自动采集与脚手架协同，控制业务侧改动面，便于团队推广与迭代

项目成果：

- 分端埋点 **SDK** 落地后，团队侧手写埋点与维护成本下降，开发效率约提升 20%
- 业务项目接入时业务代码改动面可控，便于在多条业务线复用与推广
- 脚手架集成埋点能力，新业务/新活动页接入路径缩短、上手成本更低

8、前端工程化脚手架平台 兔展科技 · 2021.05 - 2021.10

项目背景：团队规范长期停留在文档里，新建项目时监控、Lint、CI 等需**手工逐项接入**，耗时长且各仓库配法不一，缺少统一的工程化起步方式。

项目描述：建设团队级前端工程化脚手架：**项目初始化、远程模板与版本管理**统一收口，把纸面规范落实为可执行的 **CLI**，一条命令拉起可开发的工程基线。

业务价值：监控、Lint、CI 等在**创建项目时即可按需接入**，缩短立项之后到可开发、可联调的前置时间；各项目接入步骤对齐，减少「各配各的」带来的协作成本。

项目职责：主导脚手架**架构设计与技术选型**，完成 **CLI** 核心、子命令体系与远程模板管理，设计交互式创建流程。

技术要点：

- **CLI 架构**：交互式 **CLI**，以策略模式组织子命令，兼顾扩展性与可维护性
- **模板与规范**：远程模板与配置统一管理、按需拉取；创建流程中可选接入监控、Lint、CI 等预设能力
- **初始化提效**：模板与 **CLI** 解耦并版本化管理；弱网或重复创建场景下复用本地缓存，缩短初始化等待

项目成果：

- 新项目从创建到本地可运行耗时由约 2 小时压缩至 10 分钟内，启动效率提升约 90%
- 规范接入由「对照文档逐项配置」转为命令内一键完成，团队执行口径一致
- 降低新项目与新人上手成本，团队通用脚手架累计支撑数十个项目初始化

9、小程序构建发布与协作系统 兔展科技 · 2020.12 - 2021.04

项目背景：小程序发版长期靠**手动上传**，版本号**各管各的**，多人协作容易串版本、互相覆盖；团队里脚本是散的，缺少从构建、上传到预览/分发的**统一入口**。

项目描述：搭建小程序**构建—上传—分发**一体化能力：以配置驱动 **CLI** 为入口，把构建、多模板上传、预览与检查串成一条命令流；版本号与发布动作集中管理，关键步骤失败可重试；另做**取码平台**，统一输出预览码与体验码，研发、测试与业务侧在同一处取码协作。

业务价值：营销/活动类发版频繁时，缩短提审前准备与联调反馈周期；版本集中管理，降低误覆盖；取码路径固定，减少反复打包与口头传码。

项目职责：负责方案设计与核心实现：技术选型，**CLI** 与测试脚本，上传/预览/检查/版本与重试全链路，以及取码平台搭建与使用说明。

技术要点：

- **发布链路：**配置化驱动构建与上传，多模板可并行；预览与检查纳入同一命令流，失败可重试
- **版本与协作：**版本号与发布记录集中管理，缓解多人并行时的冲突与误覆盖
- **分发协作：**取码平台统一输出预览码/体验码，支撑联调、验收与企业微信等场景

项目成果：

- 构建与发布流程自动化，开发效率约提升 15%
- 版本与协作冲突缓解，协作效率约提升 60%
- 取码平台提升联调与企业微信侧效率约 30%